

Pandas Basics — Part 1

Introduction to Pandas · Loading and Inspecting Data · Dataset: tips.csv

1. What Pandas actually is

You already know NumPy. A NumPy array is a grid of numbers — fast, but it has no idea what those numbers *mean*. Row 1, column 1 is just a number sitting in a box.

```
import numpy as np

arr = np.array([[16.99, 1.01], [10.34, 1.66], [21.01, 3.50]])
print(arr[1, 1])      # 1.66
```

1.66 of what? A bill? A tip? A discount? The array cannot tell you.

Pandas fixes exactly one thing: it puts names on the rows and columns.

```
import pandas as pd

tbl = pd.DataFrame(arr, columns=['total_bill', 'tip'])
print(tbl.loc[1, 'tip'])    # 1.66 - and now you know it's a tip
```

That is the whole idea. Everything else in Pandas is a consequence of it.

Pandas is built *on top of* NumPy. It has not replaced it — NumPy is still running underneath, doing the actual maths.

2. Series — one labelled column

A **Series** is one-dimensional data with an index attached.

```
s1 = pd.Series([4200, 5100, 8900, 7600])
print(s1)
```

```
0    4200
1    5100
2    8900
3    7600
dtype: int64
```

The left column is the **index**. By default it is 0, 1, 2, 3 — but it does not have to be:

```
sales = pd.Series([4200, 5100, 8900, 7600],
                  index=['Thur', 'Fri', 'Sat', 'Sun'])
print(sales)
print(sales['Sat'])    # 8900
```

Now you look up data **by name** instead of counting positions. That is the entire advantage over a NumPy array.

Useful attributes

```
sales.index    # the labels
sales.values    # the NumPy array underneath
sales.dtype    # int64
sales.shape    # (4,) <- ONE number, because a Series is 1D
```

Remember shape giving a single number. It comes back later.

3. DataFrame — the table

A **DataFrame** is two-dimensional: several Series placed side by side, all sharing the same index.

```
df.shape      # (244, 7)    <- Two numbers, because it's 2D
df.index      # RangeIndex from 0 to 243
df.columns    # the 7 column names
```

Proof that a DataFrame really is made of Series:

```
type(df['tip'])    # Series
df['tip'].shape    # (244,)
```

Pull one column out of a DataFrame and you get a Series. There is no third structure to learn — just Series and DataFrame.

4. Building a DataFrame from a dictionary

The most common way to create one by hand.

```
data = {
    'table': [1, 2, 3, 4],
    'amount': [450, 1200, 300, 890],
    'waiter': ['Ravi', 'Sunita', 'Ravi', 'Amit']
}

orders = pd.DataFrame(data)
print(orders)
print(orders.shape)    # (4, 3)
```

The rule: each dictionary **key** becomes a column name, and each **value** becomes that column's data.

You can give your own row labels too:

```
orders2 = pd.DataFrame(data, index=['a', 'b', 'c', 'd'])
```

Watch out. Every list must be the same length. Three lists of 4, 4 and 3 will raise an error.

5. The NumPy array underneath

Pandas keeps a NumPy array inside every DataFrame. You can pull it out:

```
nums = df[['total_bill', 'tip']].to_numpy()
print(nums.dtype)          # float64 - clean numeric array
```

Now try it on a table that has **both numbers and text**:

```
mixed = orders.to_numpy()
print(mixed.dtype)         # object    <- look at this
print(mixed)
```

A NumPy array can hold only ONE data type. When the table contains numbers *and* text, everything is forced into object — and the numbers stop behaving like numbers:

```
mixed.mean()              # ERROR
```

This is why Pandas exists. Real data is always mixed — names, dates, categories, numbers. NumPy cannot hold that in one array. Pandas gives each column its own type, so a text column and a numeric column can live in the same table without interfering.

`.values` and `.to_numpy()` do the same job. `.to_numpy()` is the preferred one now.

6. Loading a CSV file

Up to now we typed data in ourselves. That is not how a real job works. Data **arrives** — exported from a system, emailed to you, downloaded from a database.

```
df = pd.read_csv('tips.csv')
print(df.shape)      # (244, 7)
```

The file must be in the same folder as your notebook, or you must give the full path.

Useful arguments

```
pd.read_csv('tips.csv', index_col=0)      # use column 0 as the index
pd.read_csv('tips.csv', nrows=10)         # only the first 10 rows
pd.read_csv('tips.csv', usecols=['tip'])   # only certain columns
```

Our dataset — tips.csv

244 restaurant bills, 7 columns.

Column	Meaning
total_bill	bill amount in dollars
tip	tip left
sex	Male / Female
smoker	Yes / No
day	Thur / Fri / Sat / Sun

Column	Meaning
time	Lunch / Dinner
size	number of people at the table

7. First look — head() and tail()

```
df.head()      # first 5 rows (default)
df.head(3)     # first 3
df.tail()      # last 5
```

Look at `tail()` carefully — the last index is **243**, not 244. With 244 rows starting at 0, the last label is 243.

8. Size and column names

```
df.shape       # (244, 7)
df.shape[0]    # 244 -> number of rows
df.shape[1]    # 7    -> number of columns
df.columns     # the column names
list(df.columns) # easier to read
df.dtypes      # the type of each column
```

No brackets. These are *attributes*, not methods. `df.shape()` raises an error.

9. info() — the health check

The single most useful line when you meet a new dataset.

```
df.info()
```

It answers **two questions at once**:

a) What type is each column?

object = text, int64 = whole numbers, float64 = decimals.

b) How many values are actually there?

This is the important one. Look at the header — RangeIndex: 244 entries. Then look at the non-null count for each column. In tips, every column shows **244 non-null**, which means nothing is missing.

That will not always be true. If a column shows 238 non-null out of 244, six values are missing — and the moment that happens, mean() on that column stops being a calculation and becomes a **decision**: do you delete those rows, or fill them in?

That question is the whole of the next class.

10. describe() — statistics in one line

```
df.describe()
```

Gives you count, mean, standard deviation, min, 25%, 50% (the median), 75% and max — for every numeric column.

Notice what is missing. Only total_bill, tip and size appear. sex, smoker, day and time are gone — because you cannot take the average of "Saturday". That is not a bug; describe() only works on things you can do maths on.

```
df.describe(include='object')    # the text columns instead
df.describe(include='all')       # everything together
```

Reading it properly. Compare the mean row with the 50% row for total_bill. They are not the same number. When the mean is larger than the median, a small number of very large bills are pulling the average up. The average alone would have misled you.

11. value_counts() — counting categories

describe() is useless for text columns. value_counts() is what you use instead.

```
df['day'].value_counts()
```

```
Sat    87
Sun    76
Thur    62
Fri    19
```

They add up to 244 — every row is counted exactly once. Always worth checking.

```
df['time'].value_counts()      # Dinner 176, Lunch 68
df['sex'].value_counts()      # Male 157, Female 87
df['smoker'].value_counts()   # No 151, Yes 93
```

Variations

```
df['day'].value_counts(normalize=True)    # proportions instead of counts
df['day'].value_counts().sort_index()     # alphabetical, not by size
```

This is the real lesson of today: **any claim someone makes about a business is one line of code away from being checked.**

12. unique() and nunique()

Three functions, three different questions:

```
df['day'].nunique()           # 4                -> HOW MANY different values
df['day'].unique()           # the day names     -> WHICH values
df['day'].value_counts()     # counts            -> HOW MANY OF EACH
```

Pick based on what you actually want to know.

```
df['size'].nunique()
df['size'].unique()
df.nunique()                # for every column at once
```

Quick reference

Command	What it gives you
<code>pd.read_csv('file.csv')</code>	loads a file into a DataFrame
<code>df.head(n) / df.tail(n)</code>	first / last n rows
<code>df.shape</code>	(rows, columns)
<code>df.columns</code>	column names
<code>df.dtypes</code>	type of each column
<code>df.info()</code>	types and missing values together
<code>df.describe()</code>	statistics for numeric columns
<code>df['col'].value_counts()</code>	count of each category
<code>df['col'].unique()</code>	which distinct values exist
<code>df['col'].nunique()</code>	how many distinct values exist
<code>df.to_numpy()</code>	the NumPy array underneath

Errors you will probably make

Mistake	Fix
<code>df.shape()</code>	<code>df.shape</code> — it's an attribute, no brackets
<code>df.info</code>	<code>df.info()</code> — this one <i>is</i> a method
<code>pd.read_csv('tips')</code>	include the extension: <code>'tips.csv'</code>
<code>df.value_counts('day')</code>	<code>df['day'].value_counts()</code> — pick the column first
File not found	the CSV must sit in the same folder as your notebook